

# Playwright Cheat Sheet

Locators · Actions · Assertions · Page Objects

## 1 LOCATORS — finding something on the page

Listed best → last resort. A locator is a saved description — it doesn't search until you act on it.

|                                                          |                                                 |
|----------------------------------------------------------|-------------------------------------------------|
| <code>getByRole('button', {name:'Add to basket'})</code> | BEST — matches how a user/screen reader sees it |
| <code>getByTestId('search-field')</code>                 | BEST when there's no clean role/label           |
| <code>getByLabel('Sort by')</code>                       | Matches via a <label> relationship              |
| <code>getByPlaceholder('Search books...')</code>         | Matches the placeholder attribute               |
| <code>getByText('Results for "Quantum"')</code>          | Matches visible page text                       |
| <code>locator('.vlist__r4k')</code>                      | LAST RESORT — CSS, breaks when classes change   |

⚠ Locators auto-wait for the element to be visible, stable, and enabled — no manual `sleep()` or `WebDriverWait` needed.

## 2 ACTIONS — doing something to it

|                                                        |                                               |
|--------------------------------------------------------|-----------------------------------------------|
| <code>locator.click()</code>                           | Click it                                      |
| <code>locator.fill('text')</code>                      | Clears + sets value directly (not keystrokes) |
| <code>locator.pressSequentially('text')</code>         | Types char-by-char (for live autocomplete)    |
| <code>locator.check()</code> / <code>.uncheck()</code> | Checkbox / radio                              |
| <code>locator.selectOption('value')</code>             | <select> dropdowns                            |
| <code>locator.hover()</code>                           | Hover over the element                        |
| <code>page.goto('/catalog')</code>                     | Navigate (lives on page, not locator)         |

⚠ Every action returns a Promise — missing `await` is the #1 beginner bug. If a test misbehaves, check for a missing `await` first.

## 3 ASSERTIONS — checking it actually worked

All assertions are prefixed with: `await expect(...)`

|                                            |                                  |
|--------------------------------------------|----------------------------------|
| <code>.toBeVisible()</code>                | Element is visible               |
| <code>.toBeHidden()</code>                 | Element is hidden                |
| <code>.toHaveText('Quantum Drift')</code>  | Exact text match                 |
| <code>.toContainText('Results for')</code> | Partial text match               |
| <code>.toHaveValue('Quantum')</code>       | Input field's current value      |
| <code>.toBeEnabled()</code>                | Not disabled                     |
| <code>.toBeChecked()</code>                | Checkbox/radio is checked        |
| <code>(page).toHaveURL(/\/catalog/)</code> | Assert on the page itself        |
| <code>.toHaveCount(14)</code>              | Assert how many elements matched |

⚠ Always put the locator INSIDE `expect()` — `expect(locator).toHaveText(...)`, never `expect(await locator.textContent()).toBe(...)`. Pulling the value out yourself loses auto-retry and causes flaky failures.

## 4 PAGE OBJECTS, STRUCTURE & CLI

### Page Object class skeleton

```
export class CatalogPage {
  constructor(private page: Page) {}

  async addToBasket(bookId: string) {
    await this.page.goto(`/catalog/${bookId}`);
    await this.page.getByTestId('add-to-basket').click();
  }
}
```

### Using it in a test

```
test('adding a book updates the basket count', async ({ page }) => {
  const catalog = new CatalogPage(page);
  await catalog.addToBasket('7'); // Quantum Drift

  await expect(page.getByTestId('basket-count')).toHaveText('1');
});
```

### Grouping & setup

```
test.describe('Catalog filtering', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('/catalog'); // runs before EVERY test below
  });

  test('filtering by genre updates the match count', async ({ page }) => { /* ... */ });
});
```

### CLI commands

|                                                  |                                               |
|--------------------------------------------------|-----------------------------------------------|
| <code>npx playwright test</code>                 | Run every test                                |
| <code>npx playwright test catalog.spec.ts</code> | Run just one file                             |
| <code>npx playwright test -g "basket"</code>     | Run tests matching a name                     |
| <code>npx playwright test --debug</code>         | Step through a test line by line              |
| <code>npx playwright test --ui</code>            | Visual test runner — pick, watch, time-travel |
| <code>npx playwright show-report</code>          | Open the HTML report from the last run        |

## 5 PUTTING IT ALL TOGETHER — A COMPLETE EXAMPLE

The same *duplicate-title* problem from the messy catalog grid — locate by role + text, then narrow with `filter()`. Every line below maps back to sections 1-4.

### CatalogPage.ts

```
import { Page, Locator } from '@playwright/test';

export class CatalogPage {
  readonly matchCount: Locator;
  readonly basketCount: Locator;

  constructor(private page: Page) {
    this.matchCount = page.getByTestId('match-count');
    this.basketCount = page.getByTestId('basket-count');
  }

  async goto() {
    await this.page.goto('/catalog');
  }

  rowsWithTitle(title: string): Locator {
    return this.page.getByRole('row').filter({ hasText: title });
  }

  async addRowToBasket(row: Locator) {
    await row.getByRole('button', { name: 'Add to basket' }).click();
  }
}
```

### catalog.spec.ts

```
import { test, expect } from '@playwright/test';
import { CatalogPage } from './pages/catalog.page';

test.describe('Catalog', () => {
  let catalog: CatalogPage;

  test.beforeEach(async ({ page }) => {
    catalog = new CatalogPage(page);
    await catalog.goto();
  });

  test('add the hardcover edition from a list with duplicate titles', async () => {
    const silentRows = catalog.rowsWithTitle('The Silent Library');
    await expect(silentRows).toHaveCount(2);

    const hardcover = silentRows.filter({ hasText: '$24.99' });
    await catalog.addRowToBasket(hardcover);

    await expect(catalog.basketCount).toHaveText('1');
  });
});
```